



Interrupts & System Calls

What is an interrupt?

Improve processor utilization by notifying the CPU of events instead of constantly checking (polling)

Interrupt Sources:

- Program
- Timer
- Input/Output
- Hardware Failure

Handling

Does three main functions:

- Stores the current state
- Handles interrupts
- Restores the original state

Other Features:

- Can be prioritized in one of two ways:
 - Sequentially
 - Nested

System Calls

Invocation:

- Request services either from the OS
 - Can be automatic or explicit

Traps:

- Software-generated interrupts due to errors or user requests

User Mode vs. Kernel Mode

Modes

User Mode: Limited access for running applications

Kernel Mode: Full access for executing OS code

Mode Switching

- Starts in kernel mode on boot
- Switches to user mode for applications
 - Back to kernel mode for interrupts and traps

Motivation

1. Protect system integrity and security
2. Manage I/O device access
3. Ensure performance trade-offs are worthwhile for security

Examples and Code

Unix System Call Example:

- `read` function demonstrates parameter passing and control transfer between modes
- Control transfers from user to kernel mode during execution

```
ssize_t read ( int file_descriptor, void *buffer , size_t count ) ;
```

Takes three params:

1. the file
2. where to read the data to
3. how many bytes to read

```
int bytesRead = read ( file , buffer , numBytes ) ;
```

System Call Summary

1. User program pushes arguments onto stack
2. User program invokes the system call
3. System call puts its identifier in the designated location
4. System call issues the `trap` function
5. OS responds to the interrupt and examines the identifier in the designated location
6. OS runs the system call handler that matches the identifier
7. When handler is finished, control exits the kernel and goes back to the system call (in user mode)
8. The system call returns control to user program

Cortex M4 Specifics

System Call Framework: Uses the SVC instruction with an integer identifier (0-255)

- This works by calling an API wrapper that calls SVC

SVC Handler Implementation:

Params:

- Identifier (int 0-255)
 - We choose the meaning of each identifier

Process:

- Run the SVC_Handler (assembly)
 - Extract the integer parameter, invoke the c function
- Run the SVC_Handler_Main (C)
 - call the individual system call corresponding to that identifier